SPARQL & Wikidata: In-Depth Explanation

Table of Contents

1. Background Concepts
2. Part 1 — Hardcoded SPARQL Query
3. Part 2 — The Entity Linking Problem
4. Part 3 — Executing the Dynamic Pipeline
5. Method 1 — Wikidata Search API (Recommended)
6. Method 2 — Finding IDs via SPARQL
7. Method 1 vs Method 2 Comparison
8. Full AI Pipeline Summary

---

1. Background Concepts

1.1 What is Wikidata?

Wikidata is a free, open, collaborative knowledge base maintained by the Wikimedia Foundation. Unlike Wikipedia, which stores information as human-readable articles, Wikidata stores information as **structured, machine-readable data** in a graph format. This makes it one of the largest open **Knowledge Graphs** in the world, containing over 100 million items.

1.2 Knowledge Graphs and Triples

All data in a knowledge graph is stored as **triples** — the most atomic unit of information:

```
Subject  →  Predicate  →  Object
```

For example:

```
Garfield  →  instance of  →  Cat
Lithuania  →  capital  →  Vilnius
Earth  →  part of  →  Solar System
```

Each triple is a single fact. Millions of triples linked together form a **graph** — a web of interconnected knowledge.

1.3 Wikidata's Identifier System

Because the same word can mean different things in different languages or contexts, Wikidata assigns every concept a **unique identifier**:

- **Items** (subjects/objects) use **Q-numbers**: `Q146` = house cat, `Q5` = human, `Q64` = Berlin.
- **Properties** (predicates/relationships) use **P-numbers**: `P31` = instance of, `P279` = subclass of, `P17` = country.

This eliminates ambiguity. The English word "cat" could refer to the animal, the musical, or a piece of machinery — but `Q146` always and exclusively means "house cat."

1.4 Key Properties

Property	Label	Meaning	Example
`P31`	instance of	"this specific thing **is a**..."	`Garfield → P31 → Q146` (Garfield is a cat)
`P279`	subclass of	"this category **is a type of**..."	`Q146 → P279 → Q729` (cat is a type of animal)
`P361`	part of	"this thing **belongs to**..."	`Q3336843 → P361 → Q544` (chapter is part of a book)
`P17`	country	"located in country..."	`Q216 → P17 → Q37` (Vilnius → country → Lithuania)

The distinction between `P31` (instance of) and `P279` (subclass of) is critical:

- **`instance of` (P31)** links a **concrete individual** to its **class**. "Garfield is an instance of cat" — Garfield is one specific cat.
- **`subclass of` (P279)** links a **class** to a **broader class**. "Cat is a subclass of animal" — the entire category of cats falls under animals.

If you want **specific named cats** (Garfield, Grumpy Cat, etc.), you use `P31`. If you want **subcategories of cats** (domestic short-hair, Persian cat, etc.), you use `P279`.

1.5 What is SPARQL?

SPARQL (SPARQL Protocol and RDF Query Language) is the standard query language for graph databases, analogous to SQL for relational databases. It works by **pattern matching** against the triples in the graph.

A SPARQL query essentially says: "Find me all triples that match this pattern."

1.6 Wikidata Prefixes

In the SPARQL queries used in this notebook, you will see shorthand prefixes:

Prefix	Full URI	Meaning
`wd:`	`http://www.wikidata.org/entity/`	A Wikidata **item** (e.g., `wd:Q146` = cat)
`wdt:`	`http://www.wikidata.org/prop/direct/`	A Wikidata **property** in its simplest "direct" form (e.g., `wdt:P31` = instance of)
`rdfs:label`	`http://www.w3.org/2000/01/rdf-schema#label`	The human-readable **name** of an entity
`wikibase:Property`	—	The RDF class that all Wikidata properties belong to

---

2. Part 1 — Hardcoded SPARQL Query

The Query

```sparql
SELECT ?item ?itemLabel
WHERE {
  ?item wdt:P31 wd:Q146.
  SERVICE wikibase:label { bd:serviceParam wikibase:language "en". }
} LIMIT 15
```

Line-by-Line Breakdown

`SELECT ?item ?itemLabel` This specifies what we want back — two columns: the entity itself (`?item`) and its English name (`?itemLabel`). Variables in SPARQL always start with `?`.

`?item wdt:P31 wd:Q146.` This is the **triple pattern** — the core of the query. It reads: "Find any `?item` where the property `P31` (instance of) points to `Q146` (house cat)." In other words: find all

things that are instances of a cat.

`SERVICE wikibase:label { bd:serviceParam wikibase:language "en". }` This is a Wikidata-specific service that automatically resolves Q-numbers into human-readable English labels. Without this line, `?itemLabel` would not exist and `?item` would just return URIs like `http://www.wikidata.org/entity/Q11840348`.

`LIMIT 15` Restricts output to 15 results. Without this, the query could return thousands of results.

The Python Code

```python
url = "https://query.wikidata.org/sparql"
headers = {
    "Accept": "application/sparql-results+json",
    "User-Agent": "MyPythonScript/1.0 (mailto:your.email@example.com)"
}
response = requests.get(url, params={"query": query}, headers=headers)
```

- `url`: The Wikidata SPARQL endpoint — a REST API that accepts SPARQL queries and returns results.
- `Accept` **header**: Tells the server we want results in JSON format (not XML or CSV).
- `User-Agent` **header**: **Required** by Wikimedia's API policy. Without a proper User-Agent, requests will be blocked with a `403 Forbidden` error. This identifies who is making the request.
- `requests.get()`: Sends the SPARQL query as a URL parameter to the endpoint.

The Problem

This approach works, but the IDs `P31` and `Q146` are **hardcoded**. A real AI system receiving the text input "show me some cats" would not know these IDs. This leads us to Part 2.

---

3. Part 2 — The Entity Linking Problem

What is Entity Linking?

Entity Linking (also called "Entity Grounding") is the process of mapping a **natural language expression** to a **unique identifier** in a knowledge base. It is one of the core challenges in Natural Language Processing (NLP) and AI.

The fundamental problem:

Human says	AI needs	Wikidata ID
"cat"	Which cat? The animal? The musical?	`Q146` (house cat)
"instance of"	Which property?	`P31`
"Paris"	Paris, France? Paris, Texas? Paris Hilton?	`Q90` (the city in France)

Without Entity Linking, an AI cannot bridge the gap between unstructured human language and structured database queries.

The Two-Step AI Pipeline

The notebook proposes a two-step pipeline that mirrors how real AI systems work:

1. **Entity Linking** — Convert natural language terms into Wikidata IDs using a search API.
2. **Dynamic Query Generation** — Inject those IDs into a SPARQL query template using Python string formatting.

---

4. Part 3 — Executing the Dynamic Pipeline

After Entity Linking resolves the IDs and the SPARQL query is dynamically generated, the final step is straightforward — send the generated query to the Wikidata endpoint and parse the results.

```python
response = requests.get(sparql_url, params={"query": dynamic_query}, headers=headers
data = response.json()
for result in data['results']['bindings']:
    print(f"- {result['itemLabel']['value']}")
```

This is identical to Part 1's execution, but the query was **generated programmatically** rather than written by hand. The entire pipeline from text input to structured output is now automated:

```
"cat" → Entity Linking → Q146 → SPARQL Generation → Query Execution → [Garfield,
Grumpy Cat, ...]
```

---

5. Method 1 — Wikidata Search API (Recommended)

How It Works

Method 1 uses the **MediaWiki Action API**, specifically the `wbsearchentities` action. This is a dedicated search endpoint built into Wikidata for exactly this purpose.

The Function

```python
def get_wikidata_id(term, language="en", entity_type="item"):
    url = "https://www.wikidata.org/w/api.php"
    params = {
        "action": "wbsearchentities",
        "search": term,
        "language": language,
        "format": "json",
        "type": entity_type
    }
    headers = {
        "User-Agent": "AILectureScript/1.0 (mailto:student@example.com)"
    }
    response = requests.get(url, params=params, headers=headers)
    data = response.json()
    if data.get("search"):
        return data["search"][0]["id"]
    else:
        return None
```

Parameter Breakdown

Parameter	Value	Purpose
`action`	`wbsearchentities`	Tells the API we want to search for entities by label
`search`	`"cat"`	The natural language term to look up
`language`	`"en"`	Search in English labels
`format`	`"json"`	Return results as JSON
`type`	`"item"` or `"property"`	Whether to search Q-items or P-properties

Why `entity_type` Matters

Wikidata has two namespaces — items (Q) and properties (P). They are separate:

- `get_wikidata_id("cat", entity_type="item")` → searches items → returns `Q146`
- `get_wikidata_id("instance of", entity_type="property")` → searches properties → returns `P31`

If you searched for `"instance of"` as an item, you might get a completely unrelated Q-number. The `type` parameter ensures you search the correct namespace.

What the API Returns

The API returns a ranked list of candidates. For the search term `"cat"`:

```json
{
  "search": [
    { "id": "Q146", "label": "cat", "description": "domestic species of small feline
    { "id": "Q2934",  "label": "cat", "description": "common name for many species..
    ...
  ]
}
```

The function takes `data["search"][0]["id"]` — the **first (most relevant) result**. This is a simplification; in a production AI system, you would typically use disambiguation logic (examining descriptions, context, etc.) to select the correct candidate.

Dynamic Query Generation

Once the IDs are resolved, Python f-strings inject them into a SPARQL template:

```python
dynamic_query = f"""
SELECT ?item ?itemLabel
WHERE {{
  ?item wdt:{instance_of_id} wd:{cat_id}.
  SERVICE wikibase:label {{ bd:serviceParam wikibase:language "en". }}
}} LIMIT 5
"""
```

Note the **double curly braces** `{{` and `}}` — this is required because Python f-strings use `{}` for variable substitution, so literal curly braces in the SPARQL syntax must be escaped by doubling them.

The result is a fully valid SPARQL query with the correct IDs inserted:

```sparql
SELECT ?item ?itemLabel
WHERE {
  ?item wdt:P31 wd:Q146.
  SERVICE wikibase:label { bd:serviceParam wikibase:language "en". }
} LIMIT 5
```

---

6. Method 2 — Finding IDs via SPARQL

How It Works

Instead of calling a separate search API, Method 2 uses **SPARQL itself** to find entity IDs by querying the `rdfs:label` field — the human-readable name attached to every entity in Wikidata.

Finding an Item ID ("cat")

```sparql
SELECT ?item WHERE {
  ?item rdfs:label "cat"@en.
} LIMIT 1
```

What this does:

- `?item rdfs:label "cat"@en.` — Find any entity whose English (`@en`) label is **exactly** the string `"cat"`.
- `LIMIT 1` — Return only the first match.

The result is a full URI: `http://www.wikidata.org/entity/Q146`. Python then extracts the ID:

```python
full_url = cat_results['results']['bindings'][0]['item']['value']
cat_id_from_sparql = full_url.split('/')[-1]  # → "Q146"
```

`split('/')[-1]` splits the URL by `/` and takes the last segment.

Finding a Property ID ("instance of")

```sparql
SELECT ?prop WHERE {
  ?prop a wikibase:Property;
        rdfs:label "instance of"@en.
} LIMIT 1
```

What this does:

- `?prop a wikibase:Property` — Restricts results to only **properties** (P-nodes). The keyword `a` is SPARQL shorthand for `rdf:type`, so this means "where `?prop` is of type Property."
- `rdfs:label "instance of"@en.` — The English label must be exactly `"instance of"`.
- The semicolon (`;`) is SPARQL syntax for chaining multiple predicates about the **same subject**. It avoids repeating `?prop` on each line.

Why the `wikibase:Property` filter is necessary:

Without it, the query would search **all** entities in Wikidata (both items and properties). The phrase "instance of" could theoretically match items too — for example, a Wikipedia article *about* the concept of "instance of." The type restriction ensures we only get the property `P31`.

Extracting the ID in Python

The extraction logic is the same as for items:

```python
full_url = instance_of_results['results']['bindings'][0]['prop']['value']
instance_of_id_from_sparql = full_url.split('/')[-1]  # → "P31"
```

Limitations of Method 2

1. **Exact string matching** — The query `rdfs:label "cat"@en` requires the label to match **character-for-character**. Searching `"Cat"` (uppercase C), `"cats"` (plural), or `"kitty"` (synonym) would return **zero results**. The Search API (Method 1) handles fuzzy matching, partial strings, and aliases.
2. **Ambiguity** — Multiple entities can have the exact same English label. `LIMIT 1` blindly takes the first one, which may not be the intended entity. Method 1's search API ranks

results by relevance and returns descriptions for disambiguation.

3. **Performance** — Scanning every `rdfs:label` across 100+ million entities is computationally expensive. The Search API uses an optimized inverted index designed specifically for fast text lookup.
4. **Language sensitivity** — The `@en` tag means only English labels are searched. If the entity lacks an English label (common for local geographic features, regional concepts, etc.), the query will fail silently.

---

7. Method 1 vs Method 2 Comparison

Criteria	Method 1 (Search API)	Method 2 (SPARQL label query)
Endpoint	`wikidata.org/w/api.php`	`query.wikidata.org/sparql`
Matching	Fuzzy — handles typos, partial strings, aliases, synonyms	Exact — must match the label string character-for-character
Disambiguation	Returns ranked list with descriptions	Returns arbitrary first match
Performance	Fast — uses optimized search index	Slower — scans label triples across all entities
Namespace filtering	Built-in `type` parameter (`item` / `property`)	Must manually add `?prop a wikibase:Property`
Robustness	Works with `"cats"`, `"Cat"`, `"kitty"`	Only works with `"cat"` (exact English label)
Use case	Production AI systems, chatbots, programmatic pipelines	When you are already in a SPARQL environment and know the exact label
Recommendation	✅ Preferred for Entity Linking	⚠️ Fragile, use with caution

---

8. Full AI Pipeline Summary

The notebook demonstrates the architecture behind how modern AI systems (Siri, Google Assistant, ChatGPT with tool use) retrieve verifiable facts from external knowledge bases:

```
┌──────────────────────────────────────────────────────────────────┐
│                    USER INPUT (Natural Language)                 │
│                       "Show me some cats"                        │
└──────────────────────────┬───────────────────────────────────────┘
                           │
                           ▼
┌──────────────────────────────────────────────────────────────────┐
│              STEP 1: ENTITY LINKING (Grounding)                  │
│                                                                  │
│   "cat"           → Wikidata Search API → Q146                   │
│   "instance of"   → Wikidata Search API → P31                    │
│                                                                  │
│   Human text is mapped to machine-readable identifiers.          │
└──────────────────────────┬───────────────────────────────────────┘
                           │
                           ▼
┌──────────────────────────────────────────────────────────────────┐
│          STEP 2: DYNAMIC QUERY GENERATION                        │
│                                                                  │
│   Python f-string injects IDs into a SPARQL template:            │
│                                                                  │
│   SELECT ?item ?itemLabel WHERE {                                │
│     ?item wdt:P31 wd:Q146.                                       │
│     SERVICE wikibase:label { ... }                               │
│   } LIMIT 5                                                     |
└──────────────────────────┬───────────────────────────────────────┘
                           │
                           ▼
┌──────────────────────────────────────────────────────────────────┐
│             STEP 3: QUERY EXECUTION                              │
│                                                                  │
│   Send SPARQL to https://query.wikidata.org/sparql               │
│   Parse JSON response                                            │
│   Extract human-readable labels                                  │
└──────────────────────────┬───────────────────────────────────────┘
                           │
                           ▼
┌──────────────────────────────────────────────────────────────────┐
│                    AI OUTPUT (Structured Answer)                 |
│                                                                  │
│   - Garfield                                                     │
│   - Grumpy Cat                                                   │
│   - Keyboard Cat                                                 |
```

```
│   - ...                                                                  │
│                                                                          │
│   Factual, verifiable, sourced from a structured Knowledge Base          │
└──────────────────────────────────────────────────────────────────────────┘
```

This pipeline is significant because it allows AI systems to answer factual questions without relying solely on memorized training data. By querying an external Knowledge Base in real time, the AI can provide **up-to-date**, **verifiable**, and **traceable** answers — directly combating the hallucination problem that plagues Large Language Models.